

CI Demo Script

Condensed run sheet for the HexaStock CI discipline walkthrough

A condensed, copy-paste-ready run sheet for demonstrating the CI discipline on this repository's `main` branch. Same idea as the smaller [ci-demo-python](#) and [ci-demo-java](#) walkthroughs, now shown on a real, production-shaped, multi-module codebase. For every step explained in full, with diagrams, see [CI_DISCIPLINE_WALKTHROUGH.md](#).

Budget about 8 minutes for the steps below. Unlike the smaller repos, the full test suite takes roughly 2 minutes on a happy path — plan the pacing around that.

Before starting

- ☐ On `main`, terminal open, the repository's **Actions** tab open in a browser tab.
- ☐ Confirm the baseline:

COPY THIS BLOCK

```
git checkout main
git pull
```

- ☐ (Optional, ~2 min) Confirm the full suite is green:

COPY THIS BLOCK

```
./mvnw clean verify
```

1. Branch, then break the build on purpose (~2 min)

COPY THIS BLOCK

```
git checkout -b demo/break-the-build
```

In `domain/src/main/java/cat/gencat/agaur/hexastock/model/portfolio/Portfolio.java`, inside `buy(...)`, comment out:

COPY THIS BLOCK

```
balance = balance.subtract(totalCost);
```

Run just the fast, infrastructure-free tests first — this is the same domain-tests-only trick used in the two smaller demos:

COPY THIS BLOCK

```
./mvnw -pl domain,application clean test
```

Expected: Tests run: 117, Failures: 3 in the domain module — PortfolioTest\$StockOperations.shouldAddHoldingAndDecreaseBalanceWhenBuying, shouldAddLotToExistingHoldingWhenBuyingMore, and shouldIncreaseBalanceAndUpdateHoldingWhenSelling. The build stops right there, in well under a second of test time — it never even reaches the modules that need Docker.

2. Push and open a pull request (~1 min)

COPY THIS BLOCK

```
git add -A
git commit -m "Demo: break the build on purpose"
git push -u origin demo/break-the-build
gh pr create --fill
```

3. Watch the check fail (~1–2 min)

- Refresh the PR page. The `build-and-test` check goes yellow → red — this takes about 30–60 seconds here, since GitHub still spins up the runner and starts the Maven reactor even though it fails fast at the `domain` module.
- **Details** on the failed check → the same three failures shown locally.
- **Merge** button: disabled — “Merging is blocked.” This is enforced by a **ruleset** on `main` (GitHub’s newer branch-protection mechanism), configured with no bypass for anyone, including administrators.
- This exact scenario was independently verified as PR [#30](#) — closed without merging, on record as proof the block works.

4. Fix it and watch it go green (~2 min)

Uncomment the line:

COPY THIS BLOCK

```
balance = balance.subtract(totalCost);
```

COPY THIS BLOCK

```
./mvnw -pl domain,application clean test    # confirm 0 failures locally first
git add -A
git commit -m "Fix: restore the cash-balance deduction"
git push
```

Refresh the PR: check reruns — now it has to run the **entire** suite (all modules, including the ones that start MySQL and MongoDB via Testcontainers), so this run takes close to **2 minutes**, not seconds. When it turns green, the **Merge** button activates.

5. Recap (~1 min)

- The two smaller demo repos show this exact mechanism in isolation, with a sub-second test suite. This repository shows the same mechanism at real scale: a multi-module Maven build, a ~2-minute full suite with Testcontainers, and a Maven reactor that fails fast when a fast module breaks but pays the full cost when only a slow module catches the bug.
- The rule that blocks the merge — a GitHub ruleset requiring the `build-and-test` check, with no bypass for anyone — is configured once, at the repository level, and then applies uniformly to every pull request from that point on.

If something stalls

- On this repository, a passing run takes close to 2 minutes (Testcontainers pulling MySQL and MongoDB images, plus SonarCloud analysis) — a failing run at the `domain` module is much faster, under a minute. Both are normal; don't wait for a fast run when the fix touches a module tested later in the reactor.
- The ruleset can be inspected or edited at <https://github.com/<owner>/HexaStock/settings/rules>.